

LAPORAN QUIZ 3

Pemrograman Web

Zenith Bites — Web Food (Fullstack)

Tech Stack: React (Vite) + Express.js + MySQL (Sequelize) + Node.js

Nama: Steve Valentino Wijaya

NIM: 242310026

1. Screenshot Code Backend

1.1 Routes routes/menuRoutes.js

```
.env JS menuRoutes.js X
backend > routes > JS menuRoutes.js > ...
1  const express = require("express");
2  const router = express.Router();
3  const {
4    getMenus,
5    getMenuById,
6    createMenu,
7    updateMenu,
8    deleteMenu,
9  } = require("../controllers/menuController");
10 const { protect } = require("../middleware/authMiddleware");
11
12 router.get("/", getMenus);
13 router.get("/:id", getMenuById);
14 router.post("/", protect, createMenu);
15 router.put("/:id", protect, updateMenu);
16 router.delete("/:id", protect, deleteMenu);
17
18 module.exports = router;
```

routes/orderRoutes.js

```
.env JS orderRoutes.js X
backend > routes > JS orderRoutes.js > ...
1  const express = require("express");
2  const router = express.Router();
3  const {
4    getOrders,
5    createOrder,
6    updateOrderStatus,
7    deleteOrder,
8  } = require("../controllers/orderController");
9  const { protect } = require("../middleware/authMiddleware");
10
11 router.get("/", protect, getOrders);
12 router.post("/", createOrder);
13 router.put("/:id", protect, updateOrderStatus);
14 router.delete("/:id", protect, deleteOrder);
15
16 module.exports = router;
```

routes/authRoutes.js

```
.env JS authRoutes.js X
backend > routes > JS authRoutes.js > ...
1  const express = require("express");
2  const router = express.Router();
3  const { loginAdmin } = require("../controllers/authController");
4
5  router.post("/login", loginAdmin);
6
7  module.exports = router;
```

1.2 Controllers

controllers/menuController.js

```
const { Menu } = require("../models");

// @desc    Ambil semua menu
// @route    GET /api/menu
// @access   Public
const getMenu = async (req, res) => {
  try {
    const menus = await Menu.findAll({ order: [["createdAt", "DESC"]] });
    res.json(menus);
  } catch (error) {
    res.status(500).json({ message: "Gagal mengambil data menu", error: error.message });
  }
};

// @desc    Ambil satu menu berdasarkan ID
// @route    GET /api/menu/:id
// @access   Public
const getMenuById = async (req, res) => {
  try {
    const menu = await Menu.findByPk(req.params.id);
    if (!menu) return res.status(404).json({ message: "Menu tidak ditemukan" });
    res.json(menu);
  } catch (error) {
    res.status(500).json({ message: "Gagal mengambil data menu", error: error.message });
  }
};

// @desc    Tambah menu baru
// @route    POST /api/menu
// @access   Private (Admin)
const createMenu = async (req, res) => {
  try {
    const { name, description, price, emoji, category, available } = req.body;

    if (!name || price === undefined) {
      return res.status(400).json({ message: "Nama dan harga menu wajib diisi" });
    }

    const menu = await Menu.create({ name, description, price, emoji, category, available });
    res.status(201).json(menu);
  } catch (error) {
    res.status(500).json({ message: "Gagal menambah menu", error: error.message });
  }
};

// @desc    Update menu
// @route    PUT /api/menu/:id
// @access   Private (Admin)
const updateMenu = async (req, res) => {
  try {
    const menu = await Menu.findByPk(req.params.id);
    if (!menu) return res.status(404).json({ message: "Menu tidak ditemukan" });

    await menu.update(req.body);
    res.json(menu);
  } catch (error) {
    res.status(500).json({ message: "Gagal mengupdate menu", error: error.message });
  }
};

// @desc    Hapus menu
// @route    DELETE /api/menu/:id
// @access   Private (Admin)
const deleteMenu = async (req, res) => {
  try {
    const menu = await Menu.findByPk(req.params.id);
    if (!menu) return res.status(404).json({ message: "Menu tidak ditemukan" });

    await menu.destroy();
    res.json({ message: "Menu berhasil dihapus" });
  } catch (error) {
    res.status(500).json({ message: "Gagal menghapus menu", error: error.message });
  }
};

module.exports = { getMenu, getMenuById, createMenu, updateMenu, deleteMenu };
```

controllers/orderController.js

```
const { Order: OrderModel, OrderItem: OrderItemModel, Menu: MenuModel } = require("../models");
const { sequelize: db } = require("../config/db");

// @desc    Ambil semua order (untuk Admin Panel)
// @route    GET /api/orders
// @access   Private (Admin)
const getOrders = async (req, res) => {
  try {
    const orders = await OrderModel.findAll({
      include: [{ model: OrderItemModel, as: "items" }],
      order: [["createdAt", "DESC"]],
    });
    res.json(orders);
  } catch (error) {
    res.status(500).json({ message: "Gagal mengambil data order", error: error.message });
  }
};

// @desc    Buat order baru (user memesan menu)
// @route    POST /api/orders
// @access   Public
const createOrder = async (req, res) => {
  const t = await db.transaction();
  try {
    const { customerName, items } = req.body;

    if (!customerName || !items || items.length === 0) {
      await t.rollback();
      return res.status(400).json({ message: "Nama pemesan dan minimal 1 item wajib diisi" });
    }

    // Validasi & hitung ulang harga dari database (bukan dari input client) agar aman
    let totalPrice = 0;
    const validatedItems = [];

    for (const item of items) {
      const menu = await MenuModel.findByPk(item.menuId);
      if (!menu) {
        await t.rollback();
        return res.status(404).json({ message: `Menu dengan id ${item.menuId} tidak ditemukan` });
      }
      const quantity = item.quantity || 1;
      totalPrice += menu.price * quantity;
      validatedItems.push({
        menuId: menu.id,
        name: menu.name,
        price: menu.price,
        quantity,
      });
    }

    const order = await OrderModel.create(
      { customerName, totalPrice, status: "Pending" },
      { transaction: t }
    );

    await OrderItemModel.bulkCreate(
      validatedItems.map((item) => ({ ...item, orderId: order.id })),
      { transaction: t }
    );

    await t.commit();

    const fullOrder = await OrderModel.findByPk(order.id, {
      include: [{ model: OrderItemModel, as: "items" }],
    });

    res.status(201).json(fullOrder);
  } catch (error) {
    await t.rollback();
    res.status(500).json({ message: "Gagal membuat order", error: error.message });
  }
};

// @desc    Update status order (Pending -> Diproses -> Selesai)
// @route    PUT /api/orders/:id
// @access   Private (Admin)
const updateOrderStatus = async (req, res) => {
  try {
    const { status } = req.body;
    const allowedStatus = ["Pending", "Diproses", "Selesai"];

    if (!allowedStatus.includes(status)) {
      return res.status(400).json({ message: "Status tidak valid" });
    }

    const order = await OrderModel.findByPk(req.params.id);
    if (!order) return res.status(404).json({ message: "Order tidak ditemukan" });

    await order.update({ status });
    res.json(order);
  } catch (error) {
    res.status(500).json({ message: "Gagal mengupdate order", error: error.message });
  }
};

// @desc    Hapus order
// @route    DELETE /api/orders/:id
// @access   Private (Admin)
const deleteOrder = async (req, res) => {
  try {
    const order = await OrderModel.findByPk(req.params.id);
    if (!order) return res.status(404).json({ message: "Order tidak ditemukan" });

    await order.destroy();
    res.json({ message: "Order berhasil dihapus" });
  } catch (error) {
    res.status(500).json({ message: "Gagal menghapus order", error: error.message });
  }
};

module.exports = { getOrders, createOrder, updateOrderStatus, deleteOrder };
```

controllers/authController.js

```
const jwt = require("jsonwebtoken");
const bcrypt = require("bcryptjs");
const { Admin } = require("../models");

// @desc    Login admin (untuk akses Admin Panel / CRUD)
// @route   POST /api/auth/login
// @access  Public
const loginAdmin = async (req, res) => {
  try {
    const { username, password } = req.body;

    if (!username || !password) {
      return res.status(400).json({ message: "Username dan password wajib diisi" });
    }

    const admin = await Admin.findOne({ where: { username } });
    if (!admin) {
      return res.status(401).json({ message: "Username atau password salah" });
    }

    const isMatch = await bcrypt.compare(password, admin.password);
    if (!isMatch) {
      return res.status(401).json({ message: "Username atau password salah" });
    }

    const token = jwt.sign(
      { id: admin.id, username: admin.username },
      process.env.JWT_SECRET,
      { expiresIn: "8h" }
    );

    res.json({
      token,
      username: admin.username,
    });
  } catch (error) {
    res.status(500).json({ message: "Terjadi kesalahan server", error: error.message });
  }
};

module.exports = { loginAdmin };
```

1.3 Models (Sequelize) models/Menu.js

```
CodeSnap JS Menu.js X
backend > models > JS Menu.js > Menu > available
1  const { DataTypes } = require("sequelize");
2  const { sequelize } = require("../config/db");
3
4  const Menu = sequelize.define("Menu", {
5    name: {
6      type: DataTypes.STRING,
7      allowNull: false,
8    },
9    description: {
10     type: DataTypes.STRING,
11     defaultValue: "",
12   },
13   price: {
14     type: DataTypes.INTEGER,
15     allowNull: false,
16     validate: { min: 0 },
17   },
18   emoji: {
19     type: DataTypes.STRING,
20     defaultValue: "🍲",
21   },
22   category: {
23     type: DataTypes.ENUM("Makanan", "Minuman"),
24     defaultValue: "Makanan",
25   },
26   available: {
27     type: DataTypes.BOOLEAN,
28     defaultValue: true,
29   },
30 });
31
32 module.exports = Menu;
33
```

models/Order.js

```
CodeSnap JS Orderjs X
backend > models > JS Orderjs > ...
1  const { DataTypes } = require("sequelize");
2  const { sequelize } = require("../config/db");
3
4  const Order = sequelize.define("Order", {
5    customerName: {
6      type: DataTypes.STRING,
7      allowNull: false,
8    },
9    totalPrice: {
10     type: DataTypes.INTEGER,
11     allowNull: false,
12   },
13   status: {
14     type: DataTypes.ENUM("Pending", "Diproses", "Selesai"),
15     defaultValue: "Pending",
16   },
17 });
18
19 module.exports = Order;
20
```

models/OrderItem.js

```
CodeSnap JS OrderItem.js X
backend > models > JS OrderItem.js > ...
1  const { DataTypes } = require("sequelize");
2  const { sequelize } = require("../config/db");
3
4  // Tabel penghubung: detail item di dalam satu Order (relasi many-to-many Order <-> Menu)
5  const OrderItem = sequelize.define("OrderItem", {
6    name: {
7      type: DataTypes.STRING,
8      allowNull: false,
9    },
10   price: {
11     type: DataTypes.INTEGER,
12     allowNull: false,
13   },
14   quantity: {
15     type: DataTypes.INTEGER,
16     allowNull: false,
17     validate: { min: 1 },
18   },
19 });
20
21 module.exports = OrderItem;
22
```

models/Admin.js

```

CodeSnap JS Admin.js X
backend > models > JS Admin.js > ...
1  const { DataTypes } = require("sequelize");
2  const { sequelize } = require("../config/db");
3
4  const Admin = sequelize.define("Admin", {
5    username: {
6      type: DataTypes.STRING,
7      allowNull: false,
8      unique: true,
9    },
10   password: {
11     type: DataTypes.STRING,
12     allowNull: false,
13   },
14 });
15
16 module.exports = Admin;
17

```

models/index.js (relasi antar tabel)

```

CodeSnap JS index.js X
backend > models > JS index.js > ...
1  const Admin = require("../Admin");
2  const Menu = require("../Menu");
3  const Order = require("../Order");
4  const OrderItem = require("../OrderItem");
5
6  // Relasi: 1 Order punya banyak OrderItem, alias "items" biar shape JSON-nya
7  // tetap sama seperti sebelumnya di frontend (order.items[...])
8  Order.hasMany(OrderItem, { as: "items", foreignKey: "orderId", onDelete: "CASCADE" });
9  OrderItem.belongsTo(Order, { foreignKey: "orderId" });
10
11 // Relasi: 1 Menu bisa muncul di banyak OrderItem
12 Menu.hasMany(OrderItem, { foreignKey: "menuId" });
13 OrderItem.belongsTo(Menu, { foreignKey: "menuId" });
14
15 module.exports = { Admin, Menu, Order, OrderItem };
16

```

1.4

Middleware & Server Entry

middleware/authMiddleware.js

```

CodeSnap JS authMiddleware.js X
backend > middleware > JS authMiddleware.js > ...
1  const jwt = require("jsonwebtoken");
2
3  // Middleware untuk memproteksi route Admin Panel (CRUD Menu & kelola Order)
4  const protect = (req, res, next) => {
5    let token;
6    const authHeader = req.headers.authorization;
7
8    if (authHeader && authHeader.startsWith("Bearer ")) {
9      token = authHeader.split(" ")[1];
10    }
11
12    if (!token) {
13      return res.status(401).json({ message: "Tidak ada token, akses ditolak" });
14    }
15
16    try {
17      const decoded = jwt.verify(token, process.env.JWT_SECRET);
18      req.admin = decoded;
19      next();
20    } catch (error) {
21      return res.status(401).json({ message: "Token tidak valid atau kadaluarsa" });
22    }
23  };
24
25 module.exports = { protect };
26

```

server.js

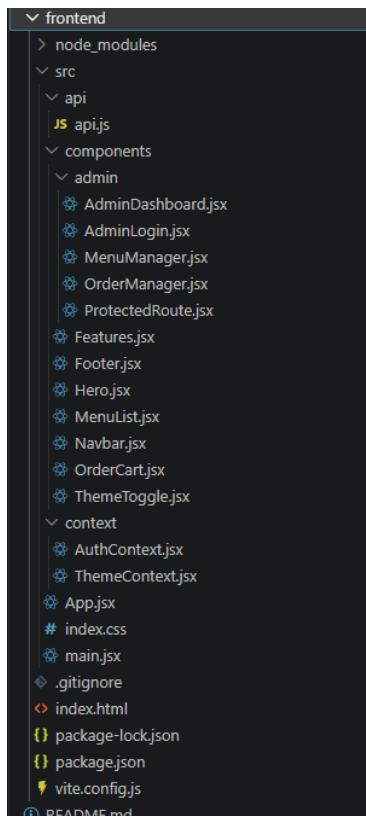

```
CodeSnap JS server.js X
backend > JS server.js > ...
1 require("dotenv").config();
2 const express = require("express");
3 const cors = require("cors");
4 const { connectDB } = require("../config/db");
5 require("../models"); // pastikan semua model & relasi ke-load sebelum sync()
6
7 const menuRoutes = require("../routes/menuRoutes");
8 const orderRoutes = require("../routes/orderRoutes");
9 const authRoutes = require("../routes/authRoutes");
10
11 connectDB();
12
13 const app = express();
14
15 app.use(cors());
16 app.use(express.json());
17
18 app.get("/", (req, res) => {
19   | res.send("Zenith Bites API sedang berjalan...");
20 });
21
22 app.use("/api/auth", authRoutes);
23 app.use("/api/menu", menuRoutes);
24 app.use("/api/orders", orderRoutes);
25
26 // Handler 404
27 app.use((req, res) => {
28   | res.status(404).json({ message: "Endpoint tidak ditemukan" });
29 });
30
31 const PORT = process.env.PORT || 5000;
32 app.listen(PORT, () => console.log(`Server berjalan di port ${PORT}`));
33
```

2. Screenshot Code Frontend

2.1 Struktur Komponen (folder src/)

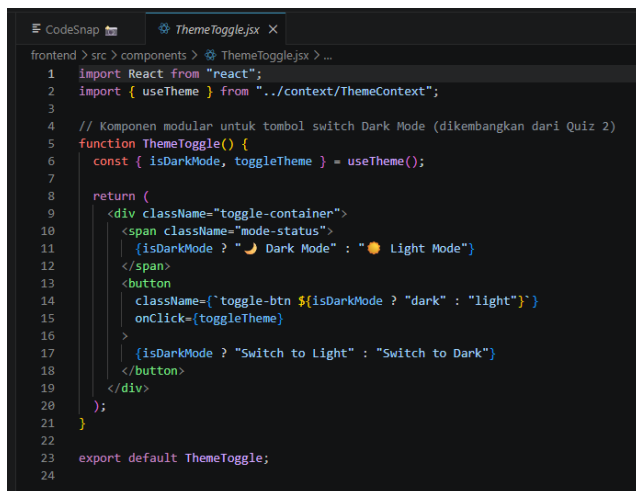
src/

- App.jsx <- routing utama (Home / Admin Login / Admin Dashboard) ■■■ main.jsx
- index.css
- api/
- ■■■ api.js <- axios instance + auto-inject token JWT
- context/
- ■■■ ThemeContext.jsx <- state global dark/light mode
- ■■■ AuthContext.jsx <- state login admin
- components/
- Navbar.jsx
- Hero.jsx
- Features.jsx
- MenuList.jsx <- fetch menu dari backend (useEffect + useState)
- OrderCart.jsx <- fitur pemesanan (keranjang -> submit order)
- Footer.jsx
- ThemeToggle.jsx <- komponen toggle Dark Mode
- admin/
- AdminLogin.jsx
- AdminDashboard.jsx
- MenuManager.jsx <- komponen CRUD Menu
- OrderManager.jsx <- komponen kelola status Order
- ProtectedRoute.jsx <- proteksi akses Admin Panel

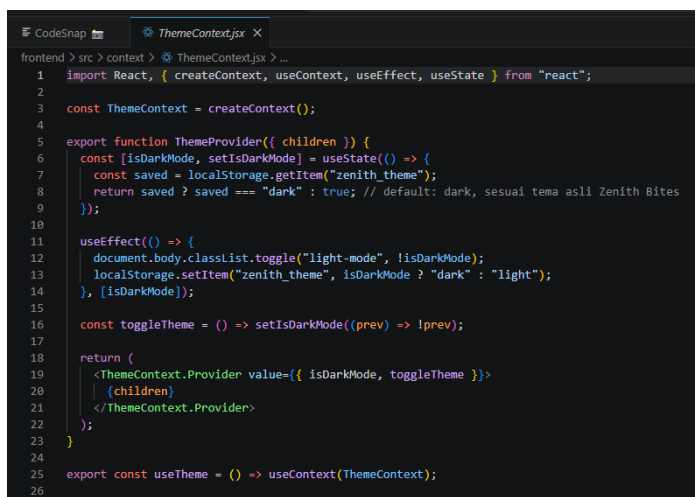


2.2 Komponen ThemeToggle (Dark Mode)

components/ThemeToggle.jsx



context/ThemeContext.jsx



2.3 Komponen CRUD / Admin Panel

components/admin/MenuManager.jsx (CRUD Menu)

```
import React, { useEffect, useState } from "react";
import api from "../../api/api";

const emptyForm = { name: "", description: "", price: "", emoji: "🍱", category: "Makanan", available: true };

// Komponen CRUD untuk mengelola Menu dari Admin Panel
function MenuManager() {
  const [menus, setMenus] = useState([]);
  const [form, setForm] = useState(emptyForm);
  const [editingId, setEditingId] = useState(null);
  const [message, setMessage] = useState(null);

  const loadMenus = async () => {
    const { data } = await api.get("/menu");
    setMenus(data);
  };

  useEffect(() => {
    loadMenus();
  }, []);

  const handleChange = (e) => {
    const { name, value, type, checked } = e.target;
    setForm((prev) => ({ ...prev, [name]: type === "checkbox" ? checked : value }));
  };

  const resetForm = () => {
    setForm(emptyForm);
    setEditingId(null);
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    setMessage(null);
    try {
      const payload = { ...form, price: Number(form.price) };
      if (editingId) {
        await api.put(`/menu/${editingId}`, payload);
        setMessage({ type: "success", text: "Menu berhasil diupdate." });
      } else {
        await api.post("/menu", payload);
        setMessage({ type: "success", text: "Menu berhasil ditambahkan." });
      }
      resetForm();
      loadMenus();
    } catch (err) {
      setMessage({ type: "error", text: err.response?.data?.message || "Gagal menyimpan menu." });
    }
  };

  const handleEdit = (menu) => {
    setEditingId(menu.id);
    setForm({
      name: menu.name,
      description: menu.description,
      price: menu.price,
      emoji: menu.emoji,
      category: menu.category,
      available: menu.available,
    });
  };

  const handleDelete = async (id) => {
    if (!window.confirm("Yakin ingin menghapus menu ini?")) return;
    try {
      await api.delete(`/menu/${id}`);
      loadMenus();
    } catch (err) {
      setMessage({ type: "error", text: "Gagal menghapus menu." });
    }
  };

  return (
    <div className="admin-panel-block">
      <h3>Kelola Menu</h3>

      <form className="admin-form" onSubmit={handleSubmit}>
        <input name="name" placeholder="Nama menu" value={form.name} onChange={handleChange} required />
        <input name="description" placeholder="Deskripsi" value={form.description} onChange={handleChange} />
        <input name="price" type="number" placeholder="Harga" value={form.price} onChange={handleChange} required />
        <input name="emoji" placeholder="Emoji (opsional)" value={form.emoji} onChange={handleChange} />
        <select name="category" value={form.category} onChange={handleChange}>
          <option value="Makanan">Makanan</option>
          <option value="Minuman">Minuman</option>
        </select>
        <label className="checkbox-label">
          <input type="checkbox" name="available" checked={form.available} onChange={handleChange} /> Tersedia
        </label>
        <div className="admin-form-actions">
          <button type="submit" className="btn-main">{editingId ? "Update Menu" : "Tambah Menu"}</button>
          <button type="button" onClick={resetForm}>Batal</button>
        </div>
      </form>

      {message && (
        <p className={`status-text ${message.type === "error" ? "error" : "success"}>{message.text}</p>
      )}

      <table className="admin-table">
        <thead>
          <tr>
            <th>Nama</th>
            <th>Kategori</th>
            <th>Harga</th>
            <th>Status</th>
            <th>Aksi</th>
          </tr>
        </thead>
        <tbody>
          {menus.map((menu) => (
            <tr key={menu.id}>
              <td>{menu.emoji} {menu.name}</td>
              <td>{menu.category}</td>
              <td>Rp {menu.price.toLocaleString("id-ID")}</td>
              <td>{menu.available ? "Tersedia" : "Habis"}</td>
              <td>
                <button onClick={() => handleEdit(menu)}>Edit</button>
                <button className="btn-remove" onClick={() => handleDelete(menu.id)}>Hapus</button>
              </td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}

export default MenuManager;
```

components/admin/OrderManager.jsx (Kelola Order)

```
1 import React, { useEffect, useState } from "react";
2 import api from "../../api/api";
3
4 const statusOptions = ["Pending", "Diproses", "Selesai"];
5
6 // Komponen untuk mengelola status Order (Pending -> Diproses -> Selesai) dari Admin Panel
7 function OrderManager() {
8   const [orders, setOrders] = useState([]);
9   const [error, setError] = useState("");
10
11   const loadOrders = async () => {
12     try {
13       const { data } = await api.get("/orders");
14       setOrders(data);
15     } catch (err) {
16       setError("Gagal memuat data order.");
17     }
18   };
19
20   useEffect(() => {
21     loadOrders();
22   }, []);
23
24   const handleStatusChange = async (id, status) => {
25     try {
26       await api.put(`/orders/${id}`, { status });
27       loadOrders();
28     } catch (err) {
29       setError("Gagal mengupdate status order.");
30     }
31   };
32
33   const handleDelete = async (id) => {
34     if (!window.confirm("Hapus order ini?")) return;
35     await api.delete(`/orders/${id}`);
36     loadOrders();
37   };
38 }
```

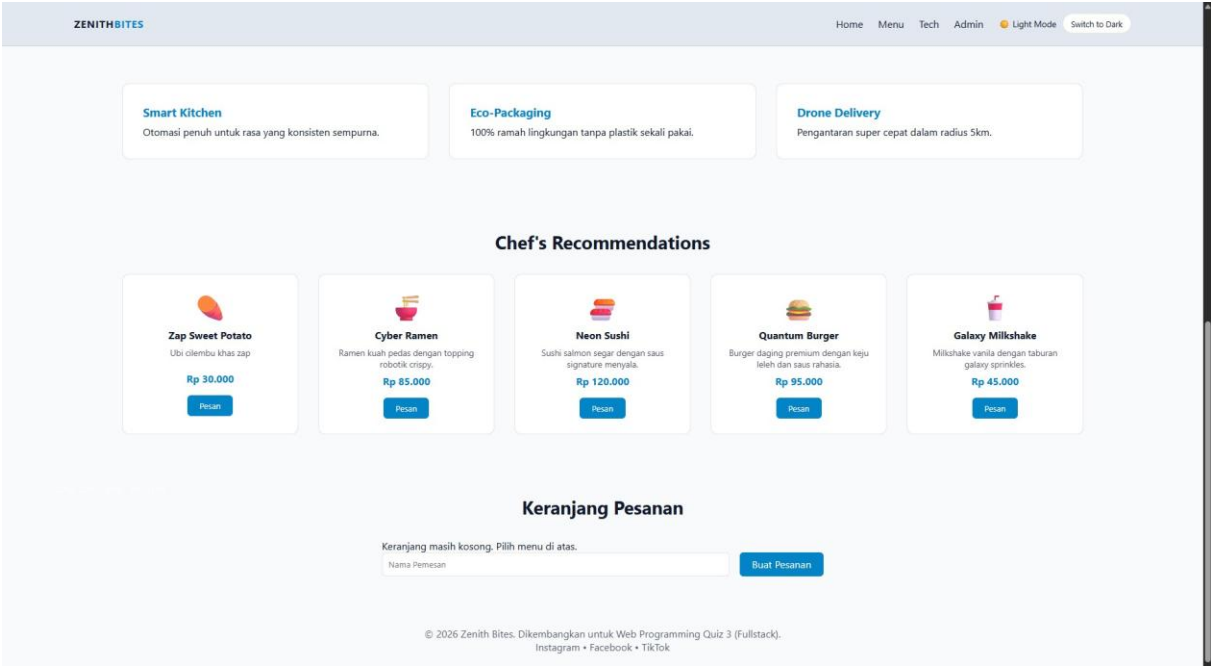
```
39 return (
40   <div className="admin-panel-block">
41     <h3>Kelola Order</h3>
42     {error && <p className="status-text error">{error}</p>}
43     <table className="admin-table">
44       <thead>
45         <tr>
46           <th>Pemesan</th>
47           <th>Item</th>
48           <th>Total</th>
49           <th>Status</th>
50           <th>Aksi</th>
51         </tr>
52       </thead>
53       <tbody>
54         {orders.map((order) => (
55           <tr key={order.id}>
56             <td>{order.customerName}</td>
57             <td>
58               {order.items.map((it) => `${it.name} x${it.quantity}`).join(", ")}
59             </td>
60             <td>Rp {order.totalPrice.toLocaleString("id-ID")}</td>
61             <td>
62               <select
63                 value={order.status}
64                 onChange={(e) => handleStatusChange(order.id, e.target.value)}
65               >
66                 {statusOptions.map((s) => (
67                   <option key={s} value={s}>{s}</option>
68                 ))}
69               </select>
70             </td>
71             <td>
72               <button className="btn-remove" onClick={() => handleDelete(order.id)}>Hapus</button>
73             </td>
74           </tr>
75         ))}
76         {orders.length === 0 && (
77           <tr><td colspan="5">Belum ada order.</td></tr>
78         )}
79       </tbody>
80     </table>
81   </div>
82 );
83 }
84
85 export default OrderManager;
```

2.4 Fetching Data dari Backend (useState + useEffect) components/MenuList.jsx

```
CodeSnap MenuList.jsx X
frontend > src > components > MenuList.jsx > ...
1 import React, { useEffect, useState } from "react";
2 import api from "../api/api";
3
4 // Komponen ini mengambil data Menu dari backend (bukan hardcoded lagi seperti Quiz 1)
5 function MenuList({ cart, onAddToCart }) {
6   const [menus, setMenus] = useState([]);
7   const [loading, setLoading] = useState(true);
8   const [error, setError] = useState("");
9
10  useEffect(() => {
11    const fetchMenus = async () => {
12      try {
13        const { data } = await api.get("/menu");
14        setMenus(data);
15      } catch (err) {
16        setError("Gagal memuat menu. Pastikan backend sedang berjalan.");
17      } finally {
18        setLoading(false);
19      }
20    };
21    fetchMenus();
22  }, []);
23
24  if (loading) return <p className="status-text">Memuat menu...</p>;
25  if (error) return <p className="status-text error">{error}</p>;
26
27  return (
28    <section id="menu" className="menu-section">
29      <h2 className="section-title">Chef's Recommendations</h2>
30      <div className="container grid">
31        {menus.length === 0 && <p className="status-text">Belum ada menu tersedia.</p>}}
32        {menus.map((menu) => (
33          <div className="menu-card" key={menu.id}>
34            <span className="emoji">{menu.emoji}</span>
35            <h4>{menu.name}</h4>
36            <p className="menu-desc">{menu.description}</p>
37            <p className="menu-price">Rp {menu.price.toLocaleString("id-ID")}</p>
38            <button onClick={() => onAddToCart(menu)} disabled={!menu.available}>
39              {menu.available ? "Pesan" : "Habis"}
40            </button>
41          </div>
42        ))}
43      </div>
44    </section>
45  );
46 }
47
48 export default MenuList;
49
```

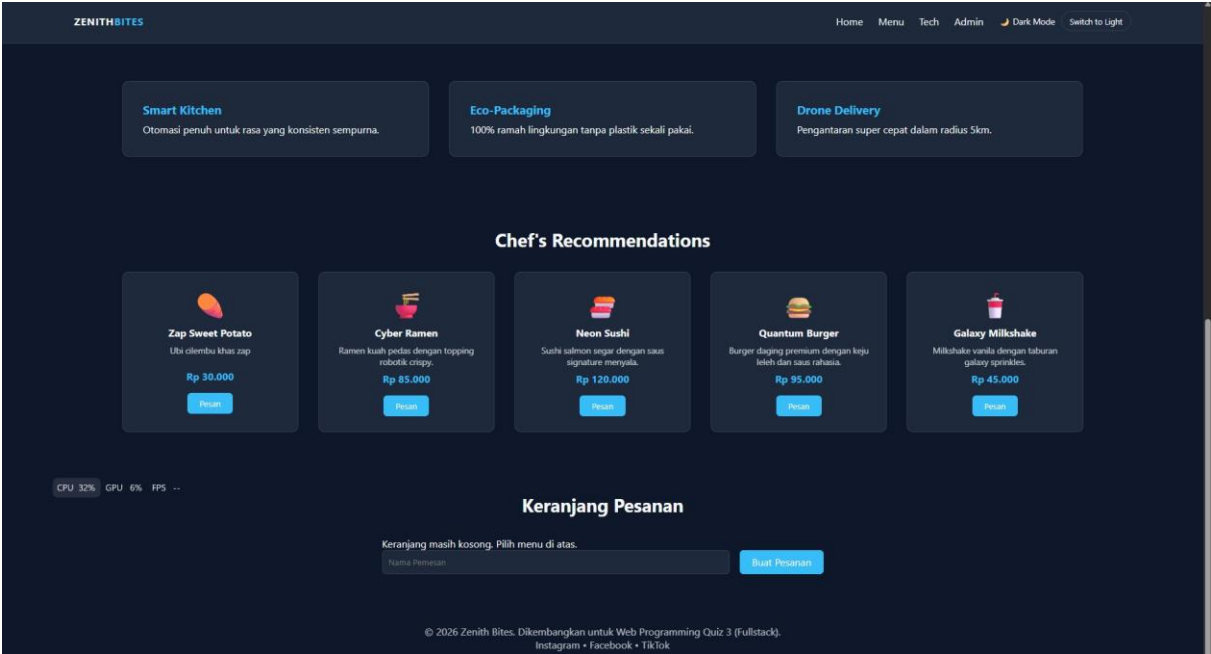
3. Screenshot Tampilan Aplikasi (Light & Dark Mode)

3.1 Halaman Utama — Light Mode



Halaman utama Zenith Bites dalam Light Mode, menu diambil dari database.

3.2 Halaman Utama — Dark Mode



Halaman utama Zenith Bites dalam Dark Mode.

3.3 Admin Panel — Light Mode

Admin Panel — Zenith Bites

Halo, adminLight ModeSwitch to DarkLogout

Kelola Menu

Nama menu

Deskripsi

Harga

Makanan

☒ Tersedia

Tambah Menu

Menu berhasil ditambahkan.

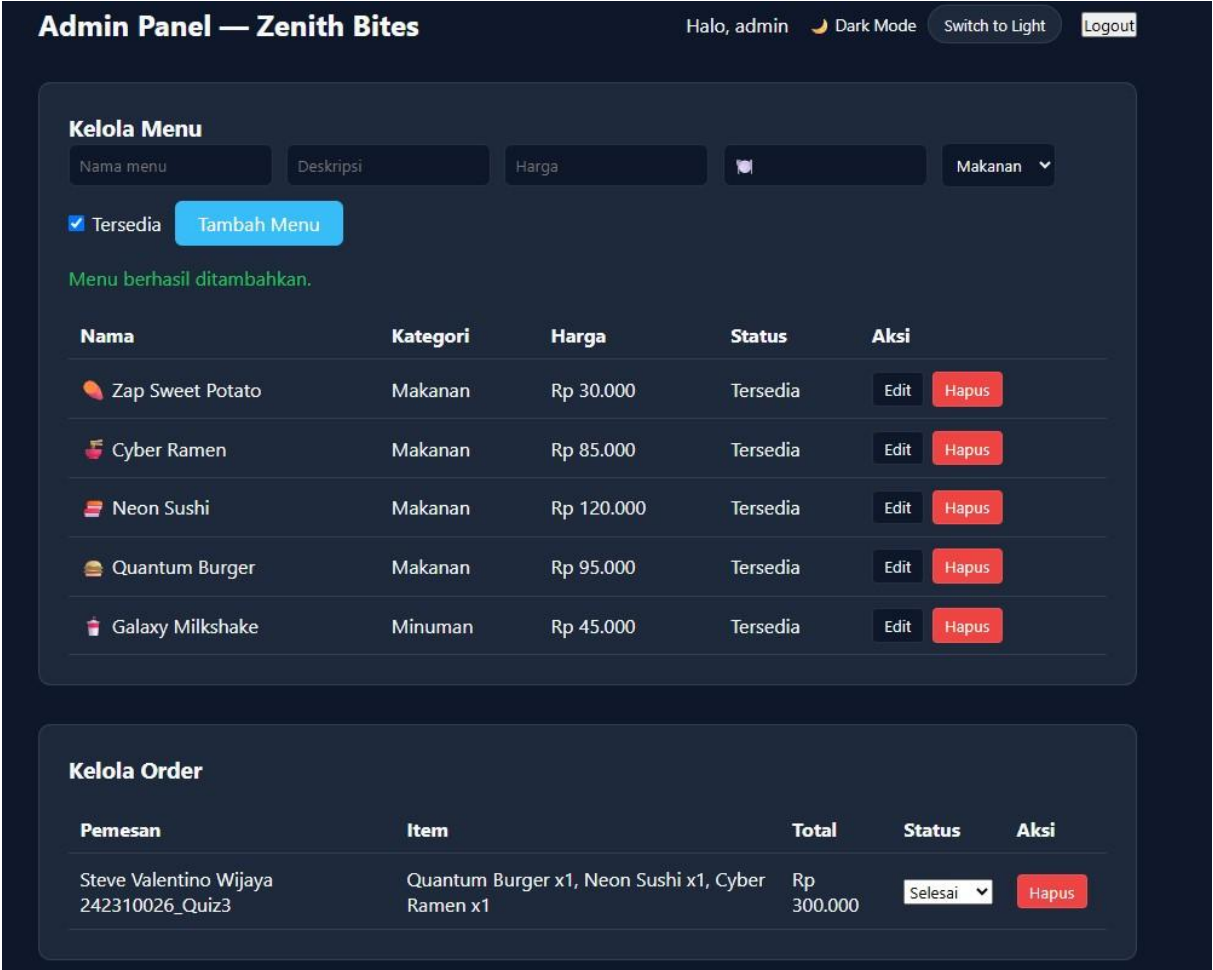
Nama	Kategori	Harga	Status	Aksi	
Zap Sweet Potato	Makanan	Rp 30.000	Tersedia	Edit	Hapus
Cyber Ramen	Makanan	Rp 85.000	Tersedia	Edit	Hapus
Neon Sushi	Makanan	Rp 120.000	Tersedia	Edit	Hapus
Quantum Burger	Makanan	Rp 95.000	Tersedia	Edit	Hapus
Galaxy Milkshake	Minuman	Rp 45.000	Tersedia	Edit	Hapus

Kelola Order

Pemesan	Item	Total	Status	Aksi
Steve Valentino Wijaya 242310026_Quiz3	Quantum Burger x1, Neon Sushi x1, Cyber Ramen x1	Rp 300.000	Selesai	Hapus

Admin Panel (kelola Menu & Order) dalam Light Mode.

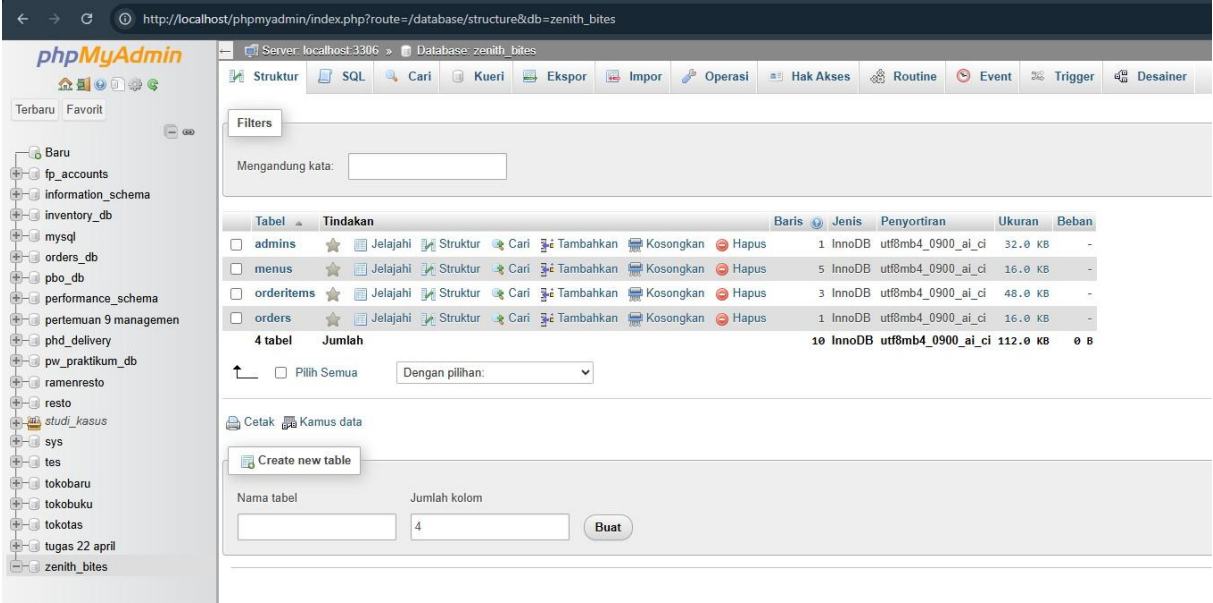
3.4 Admin Panel — Dark Mode



Admin Panel dalam Dark Mode, status mode ditampilkan di navbar.

4. Screenshot Data Tersimpan di MySQL

4.1 Struktur Tabel di phpMyAdmin



Database zenith_bites berisi 4 tabel: admins, menus, orders, orderitems.

4.2 Data pada Tabel Menu

Terbaru

Favorit

Baru

fp_accounts

information_schema

inventory_db

mysql

orders_db

pbo_db

performance_schema

pertemuan 9 managemen

phd_delivery

pw_praktikum_db

ramenresto

resto

studi_kasus

sys

tes

tokobaru

tokobuku

tokotas

tugas 22 april

zenith_bites

Server: localhost:3306

Database: zenith_bites

Tabel: menu

Jelajahi

Struktur

SQL

Cari

Tambahkan

Ekspor

Impor

Hak Akses

Operasi

Trigger

Menampilkan baris 0 - 4 (total 5, Pencarian dilakukan dalam 0.0010 detik)

SELECT * FROM `menu`

Profil [Edit dikotak] [Ubah] [Jelaskan SQL] [Buat kode PHP] [Segarkan]

Tampilkan semua

Jumlah baris: 25

Saring baris: Cari di tabel ini

Sort by key: Tidak ada

Extra options

Id

name

description

price

emoji

category

available

createdAt

updatedAt

Ubah

Salin

Hapus

1

Cyber Ramen

Ramen kuah pedas dengan topping robotik crispy.

85000

Makanan

1

2026-07-24 10:51:53

2026-07-24 10:51:53

Ubah

Salin

Hapus

2

Neon Sushi

Sushi salmon segar dengan saus signature menyala.

120000

Makanan

1

2026-07-24 10:51:53

2026-07-24 10:51:53

Ubah

Salin

Hapus

3

Quantum Burger

Burger daging premium dengan keju leleh dan saus r...

95000

Makanan

1

2026-07-24 10:51:53

2026-07-24 10:51:53

Ubah

Salin

Hapus

4

Galaxy Milkshake

Milkshake vanilla dengan taburan galaxy sprinkles.

45000

Minuman

1

2026-07-24 10:51:53

2026-07-24 10:51:53

Ubah

Salin

Hapus

5

Zap Sweet Potato

Ubi cilembu khas zap

30000

Makanan

1

2026-07-24 10:55:28

2026-07-24 10:55:28

Pilih Semua

Dengan pilihan:

Ubah

Salin

Hapus

Ekspor

Tampilkan semua

Jumlah baris: 25

Saring baris: Cari di tabel ini

Sort by key: Tidak ada

Operasi hasil kueri

Cetak

Salin ke clipboard

Ekspor

Tampilkan bagan

Buat tampilan

Data menu tersimpan di tabel menu, hasil CRUD dari Admin Panel.